

University of Victoria
Faculty of Engineering
Summer 2026 ENGR 446 Final Report

**Design and Analysis of a Pipelined RTL Hardware Accelerator
for Real-Time Sobel Edge Detection**

Robin Zerr
V01013510
ENGR 446
Electrical Engineering
Email: robinzerr@uvic.ca

July 20, 2026

In partial fulfillment of the requirements of the B.Eng. Degree

Monty Raisinghani, Instructor
ENGR 446
Faculty of Engineering
University of Victoria
Victoria, BC

July 20, 2026

Re: Design and Analysis of a Pipelined RTL Hardware Accelerator for Real-Time Sobel Edge Detection

Dear Mr. Raisinghani,

Attached is the technical report “Design and Analysis of a Pipelined RTL Hardware Accelerator for Real-Time Sobel Edge Detection,” submitted in partial fulfillment of the requirements of ENGR 446 in the electrical engineering program.

I am a fourth-year electrical engineering student, and this project draws on my coursework and co-op experience in digital systems design. The report asks whether a dedicated hardware accelerator can guarantee real-time Sobel edge detection on a streaming video source, something a general-purpose processor cannot promise. It compares a sequential and a pipelined register-transfer-level architecture, selects the pipelined design through a weighted assessment, and analyses its throughput, latency, arithmetic, resource usage, and power. The design was implemented in VHDL, verified in behavioural simulation, and synthesized in Xilinx Vivado to confirm the hardware structure. The analysis is limited to grayscale images and the standard 3×3 Sobel operator.

The principal recommendation is to proceed with the pipelined architecture. It sustained one output pixel per clock cycle in simulation, providing roughly ten times the required real-time pixel rate at a 100 MHz clock, and synthesis confirms it uses no DSP blocks.

Thank you for taking the time to review this report. I look forward to your feedback.

Sincerely,

Robin Zerr
V01013510

Table of Contents

Executive Summary.....	iii
Glossary.....	iv
List of Symbols.....	v
1.0 Introduction.....	1
2.0 Background.....	1
3.0 Objective.....	2
3.1 Problem.....	2
3.2 Purpose.....	2
3.3 Aims.....	2
3.4 Limitations.....	2
4.0 Potential Solutions.....	3
4.1 Sequential RTL Filter.....	3
4.2 Pipelined RTL Filter.....	3
5.0 Initial Assessment.....	3
6.0 Proposed Solution.....	4
7.0 Engineering Analysis.....	6
7.1 Real-Time Throughput Requirement.....	6
7.2 Sequential RTL Throughput.....	6
7.3 Pipelined RTL Throughput.....	7
7.4 Latency Analysis.....	7
7.5 Sobel Arithmetic Analysis.....	8
7.6 Resource Usage Analysis.....	8
7.7 Verification Results.....	10
7.8 Sensitivity and Limitations of the Analysis.....	11
8.0 Discussion.....	12
9.0 Conclusions.....	13
10.0 Recommendations.....	13
References.....	14
Appendix A: Supporting Calculations.....	A-1

List of Figures

Figure 1: Block diagram of the pipelined Sobel accelerator.....	5
Figure 2: The 3×3 pixel window and the Sobel kernels.....	5
Figure 3: Pipeline timing.....	5

Figure 4: Synthesized netlist of the sobel_core arithmetic pipeline.....9

Figure 5: Sobel verification result for a 128×72 input image.....10

Figure 6: Sobel verification result for a 228×128 input image.....11

List of Tables

Table 1: Weighted decision matrix for selecting the RTL Sobel architecture.....3

Executive Summary

Real-time embedded vision systems must process every pixel of a continuous video stream within a fixed time budget. General-purpose processors execute image filtering as a sequence of instructions whose timing varies with memory and cache behaviour, so they cannot guarantee the throughput and bounded latency that sustained operation requires. This report determines the most suitable register-transfer-level (RTL) hardware architecture for a real-time Sobel edge-detection accelerator operating on a streaming grayscale image source.

Two candidate architectures were compared using a weighted decision matrix. A sequential filter shares arithmetic hardware over several clock cycles per pixel, while a pipelined filter uses line buffers and shift registers to produce one output pixel per clock cycle. The pipelined design won the assessment, was implemented in VHDL, and was analysed for throughput, latency, arithmetic behaviour, resource usage, and power. The design was then verified in behavioural simulation and synthesized in Xilinx Vivado.

The application requires a sustained rate of 9.216 million pixels per second for 640×480 video at 30 frames per second. In simulation the pipelined design sustained one output pixel per clock cycle, or 100 million pixels per second at a 100 MHz clock, roughly ten times the requirement. The simulations produced the expected output frames and clean edge images, and the measured pipeline latency of 1,285 clock cycles (12.85 μ s) agreed exactly with the analytical prediction. Synthesis confirms the intended hardware. The arithmetic maps to carry-chain adders and simple logic with zero DSP blocks, the two row buffers fit in block RAM, and a worst-case vectorless power estimate gives 5.4 W with most of the power in logic and I/O rather than memory.

The pipelined RTL filter is therefore recommended. Its throughput margin already covers full-HD video at 30 frames per second, and the line-buffer streaming pattern generalises to other convolution kernels, making the accelerator a reusable building block for embedded vision. Recommended next steps are to complete implementation timing sign-off, refine the power estimate with real switching activity, and add a standard streaming interface.

Glossary

Term	Definition
ASIC	Application-specific integrated circuit, a custom-fabricated chip designed for a particular function.
AXI-Stream	A streaming data interface standard using a ready/valid handshake to transfer data with back-pressure.
BRAM	Block RAM, dedicated on-chip memory blocks within an FPGA.
Convolution (2D)	An operation in which a kernel is slid across an image and combined with local pixels by multiply-accumulate.
CPU	Central processing unit, a general-purpose instruction-executing processor.
DSP block	Digital signal processing block, a hardened FPGA multiply-accumulate resource.
Edge detection	Identification of intensity boundaries in an image, typically via gradient estimation.
FPGA	Field-programmable gate array, a reconfigurable logic device.
Flip-flop (FF)	A single-bit clocked storage element used to register signals.
Latency	The delay between data entering and the corresponding result leaving a circuit.
Line buffer	On-chip memory storing one image row to provide vertically aligned pixels for windowing.
LUT	Look-up table, the basic combinational logic element of an FPGA.
Netlist	The gate-level circuit that synthesis produces from the RTL description.
Pipelining	Partitioning a datapath into registered stages so multiple operations proceed concurrently.
RTL	Register-transfer level, a hardware description abstraction in terms of registers and logic.
Sobel operator	A pair of 3×3 kernels estimating horizontal and vertical image gradients.
Throughput	The rate at which results are produced, here in output pixels per second.
Timing closure	The state in which a synthesised circuit meets its target clock-period constraints.
Vectorless power analysis	A power estimate based on assumed switching activity rather than recorded simulation data.

List of Symbols

Symbol	Description	Units
W	Image width	pixels
H	Image height	pixels
F	Frame rate	frames/s
Npx	Pixels per frame	pixels
Rpx	Required sustained pixel rate	pixels/s
fclk	Clock frequency	Hz
CPP	Clock cycles per output pixel	cycles
Tseq	Sequential-architecture throughput	pixels/s
Tpipe	Pipelined-architecture throughput	pixels/s
L	Total initial latency	cycles
Dw	Window-formation delay	cycles
Da	Arithmetic pipeline depth	cycles
Gx, Gy	Horizontal and vertical Sobel gradients	none
M	Approximate edge magnitude	none
p _{ij}	Pixel intensity at window row i, column j	none
S	Total weighted decision-matrix score	none
w _i , r _i	Decision-matrix weight and score for criterion i	none

1.0 Introduction

Embedded vision systems are deployed in robotics, automotive driver assistance, industrial inspection, and consumer devices, where they must analyse a continuous stream of camera images in real time. A common early step in such systems is edge detection, which identifies the boundaries between regions of an image and reduces the data that later stages must process. The Sobel operator is a widely used edge detector based on two-dimensional (2D) convolution with a small fixed kernel.

The engineering problem addressed in this report is that general-purpose processors may not guarantee the deterministic throughput and bounded latency required to apply Sobel filtering to a video stream in real time. A dedicated hardware accelerator described at the register-transfer level (RTL) and implemented on a field-programmable gate array (FPGA) can provide deterministic, high-throughput processing, but the designer must choose an appropriate architecture.

This report determines the most suitable RTL architecture for a real-time 3×3 Sobel accelerator. Section 2.0 provides background on 2D convolution, Sobel filtering, and hardware acceleration. Section 3.0 states the objective, aims, and limitations. Section 4.0 describes two candidate architectures, and Section 5.0 presents the initial assessment used to select between them. Section 6.0 details the proposed pipelined solution, Section 7.0 presents the engineering analysis together with the simulation and synthesis results, and Section 8.0 discusses the findings. Sections 9.0 and 10.0 give the conclusions and recommendations.

2.0 Background

Two-dimensional convolution slides a small coefficient matrix, called a kernel, across an image and computes each output pixel as the weighted sum of a local neighbourhood of input pixels. The same multiply-accumulate pattern repeats for every pixel, making the operation computationally regular but data-intensive, and therefore well suited to hardware acceleration. The kernel coefficients determine the filtering behaviour, including blurring, sharpening, and gradient estimation [1].

Sobel edge detection applies two 3×3 kernels to estimate the horizontal and vertical intensity gradients, then combines them into an edge-magnitude image. It is favoured in embedded vision because it is computationally inexpensive, provides reasonable noise immunity through a built-in smoothing term, and maps efficiently onto fixed-function arithmetic. Its small fixed kernel also makes it a representative case for studying convolution accelerators in general [2].

Real-time processing imposes a hard timing constraint. Every pixel of every frame must be processed within a fixed budget. A central processing unit (CPU) executes convolution as instructions whose timing is affected by memory access and cache behaviour, introducing variability that can break the timing budget. FPGAs and RTL architectures instead implement the

convolution as dedicated logic, forming a spatial datapath that processes pixels deterministically as they stream through, trading silicon area for throughput [3].

Four quantities dominate such designs. Throughput measures output pixels per unit time and determines whether the frame rate can be sustained. Latency measures the delay from input to output and matters for closed-loop systems. Resource usage measures the look-up tables (LUTs), flip-flops (FFs), digital signal processing (DSP) blocks, and block RAM (BRAM) consumed, constraining cost. Timing closure determines whether the circuit can meet its target clock period. A design that cannot close timing cannot run at the intended frequency regardless of its theoretical merit.

3.0 Objective

3.1 Problem

Real-time Sobel edge detection requires deterministic, sustained processing of a continuous pixel stream. A processor-based implementation may not provide guaranteed throughput and bounded latency at standard video resolutions and frame rates because instruction execution and memory access introduce variability. A dedicated RTL accelerator is required, but the most suitable architecture is not obvious.

3.2 Purpose

The purpose of this report is to determine the most suitable RTL hardware architecture for a real-time 3×3 Sobel edge-detection accelerator processing a streaming grayscale image source.

3.3 Aims

The aims of the project are to:

1. Support at least 640×480 grayscale video at 30 frames per second, a sustained rate of 9.216 million pixels per second.
2. Compare a sequential and a pipelined RTL architecture using cycles per output pixel as the primary throughput metric.
3. Estimate or measure FPGA resource usage (LUTs, flip-flops, DSP blocks, BRAM) using Xilinx Vivado synthesis [5].
4. Determine whether the selected design can meet a clock frequency sufficient for real-time processing of the target stream.
5. Verify the RTL output in simulation using stored test images across multiple frame sizes.

3.4 Limitations

- Only grayscale (single-channel) images are processed. RGB colour is excluded.
- Only the standard 3×3 Sobel operator is considered.

- No physical camera interface is required. Inputs are stored images or test vectors.
- No full ASIC layout is performed and no neural-network inference is included.
- The implementation is verified in simulation using stored inputs rather than live video.

4.0 Potential Solutions

4.1 Sequential RTL Filter

The sequential RTL filter assembles a 3×3 pixel window and performs the Sobel arithmetic over several clock cycles using a small number of shared arithmetic units sequenced by a finite-state machine. Because few operations occur in parallel, it uses modest logic and is straightforward to design and verify. Its limitation is throughput. Each output pixel requires multiple clock cycles, so the sustained rate is the clock frequency divided by the cycles per pixel, which may be insufficient at higher resolutions or frame rates.

4.2 Pipelined RTL Filter

The pipelined RTL filter uses two line buffers and shift registers to continuously assemble a 3×3 window from the incoming stream, and partitions the Sobel arithmetic into registered stages so that, after an initial fill delay, one output pixel is produced per clock cycle [3]. It is more complex and consumes more memory and registers, but it directly targets the high-throughput, low-latency streaming nature of the problem.

5.0 Initial Assessment

The two architectures are compared using a weighted decision matrix. Seven criteria, mapped onto the project aims and the engineering quantities of Section 2.0, are each weighted by importance and scored from 1 (poor) to 5 (excellent). The total weighted score for each candidate is given by Equation (1).

$$S = \sum (w_i \times r_i) \quad (1)$$

In Equation (1), S is the total weighted score, w_i is the weight of criterion i , and r_i is the score awarded to the candidate for criterion i , summed over all seven criteria. Throughput and latency receive the highest weights because the central problem is sustaining a deterministic real-time pixel rate. Timing feasibility is weighted highly because a design that cannot close timing cannot meet the rate. Resource usage carries moderate weight, and implementation and verification complexity carry lower weight because they affect development effort rather than delivered performance.

Table 1: Weighted decision matrix for selecting the RTL Sobel architecture.

Criterion	Weight	Seq. r	Seq. w×r	Pipe. r	Pipe. w×r
Throughput	0.25	2	0.50	5	1.25

Latency	0.15	3	0.45	4	0.60
Hardware resource usage	0.15	5	0.75	3	0.45
Timing feasibility	0.15	4	0.60	4	0.60
Implementation complexity	0.10	5	0.50	2	0.20
Verification complexity	0.10	4	0.40	3	0.30
Relevance to industry skills	0.10	3	0.30	5	0.50
Total weighted score (S)	1.00		3.50		3.90

As shown in Table 1, the pipelined RTL filter achieves the higher total weighted score, driven by its superior throughput. The sequential design scores well on resource usage and simplicity but is penalised on throughput, the criterion most directly tied to the engineering problem. The pipelined RTL filter was therefore selected as the proposed solution, and the engineering analysis of Section 7.0 tests that selection quantitatively.

6.0 Proposed Solution

The proposed solution is a pipelined RTL Sobel edge-detection accelerator that processes a continuous stream of 8-bit grayscale pixels and produces one filtered output pixel per clock cycle in steady state. The design is written in VHDL and is organised into the hierarchy shown in Figure 1. A window-forming stage built from two line buffers and a bank of shift registers feeds a two-stage arithmetic pipeline. Every stage is registered, so a new pixel is accepted on each clock edge once the pipeline has filled [3].

Incoming pixels enter one per clock cycle. The two line buffers each store one full image row, holding the two previous rows so that the three vertically aligned pixels of the current window column are available at the same time. Three 3-tap shift registers then provide the horizontal taps, presenting all nine pixels of the window (p_{00} through p_{22}) to the arithmetic stages in parallel. The window labelling and the two Sobel kernels are shown in Figure 2 [1].

The arithmetic part of the design evaluates each 3×3 group of pixels and measures how much the image changes from left to right and from top to bottom. These two gradients are combined to estimate the edge strength at that point. If the result is larger than an 8-bit pixel can store, it is capped at the maximum value. The design also tracks whether each output pixel is valid. At start-up the buffers need time to fill, and pixels along the outside border cannot be processed because they do not have a full 3×3 neighbourhood. For this reason the output image is two pixels smaller in each direction. Figure 3 shows this timing. After a short start-up delay the design produces one output pixel every clock cycle. This is the benefit of a pipelined design.

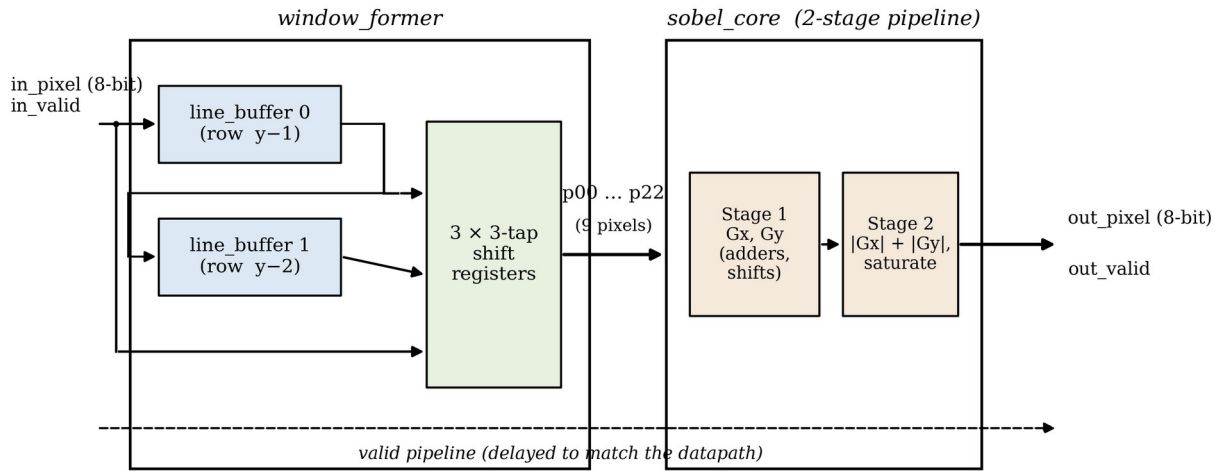


Figure 1: Block diagram of the pipelined Sobel accelerator.

3×3 pixel window			Gx kernel (horizontal)			Gy kernel (vertical)		
p00	p01	p02	-1	0	+1	-1	-2	-1
p10	p11	p12	-2	0	+2	0	0	0
p20	p21	p22	-1	0	+1	+1	+2	+1

Figure 2: The 3×3 pixel window and the horizontal (Gx) and vertical (Gy) Sobel kernels.

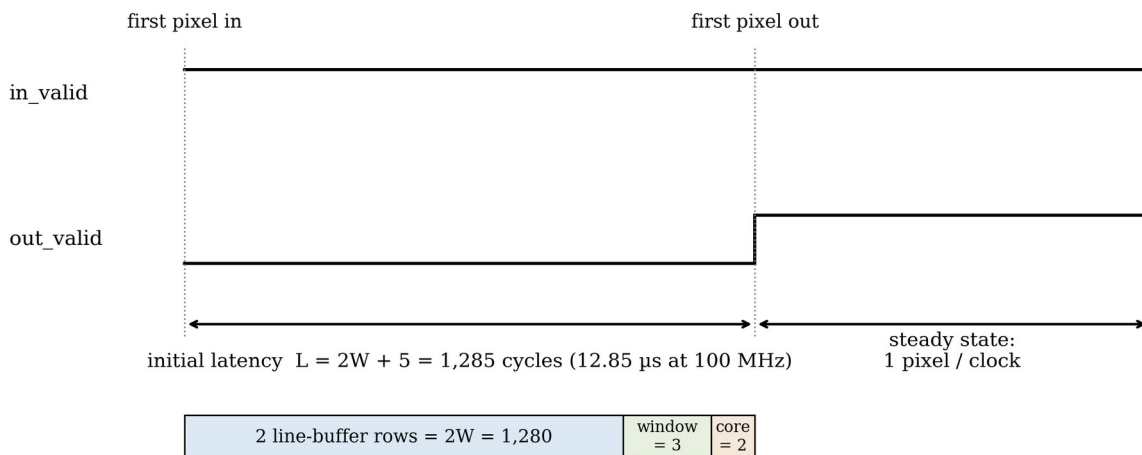


Figure 3: Pipeline timing, showing the fill latency.

7.0 Engineering Analysis

The analysis proceeds from the application requirement to the measured behaviour of the design. Section 7.1 establishes the pixel rate the accelerator must sustain. Sections 7.2 and 7.3 compare the throughput of the sequential and pipelined architectures. Section 7.4 derives and then measures the pipeline latency, and Section 7.5 examines the Sobel arithmetic and its bit widths. Sections 7.6 through 7.8 cover resource usage, verification results, and the sensitivity and limitations of the analysis. All simulation results were obtained with the Vivado simulator (VHDL-2008) using a file-driven testbench and a 100 MHz testbench clock [4].

7.1 Real-Time Throughput Requirement

The first requirement is the sustained pixel rate that the accelerator must support. The number of pixels in one frame is the product of the image width and height, as given in Equation (2).

$$N_{px} = W \times H \quad (2)$$

In Equation (2), N_{px} is the number of pixels in one frame, W is the image width in pixels, and H is the image height in pixels. The required sustained pixel rate is the pixels per frame multiplied by the frame rate, as in Equation (3).

$$R_{px} = W \times H \times F \quad (3)$$

In Equation (3), R_{px} is the required pixel rate in pixels per second and F is the frame rate in frames per second. For the target of 640×480 grayscale video at 30 frames per second, Equation (3) gives a required rate of $640 \times 480 \times 30 = 9.216 \times 10^6$ pixels per second. Any candidate architecture must sustain at least this rate to meet the project aim.

7.2 Sequential RTL Throughput

A sequential architecture shares a small number of arithmetic units across the Sobel computation, so it needs several clock cycles for each output pixel. The exact count depends on how many multiply-accumulate and absolute-value steps are serialised. The sustained throughput of such a design is given by Equation (4).

$$T_{seq} = f_{clk} / CPP \quad (4)$$

In Equation (4), T_{seq} is the sequential throughput in output pixels per second, f_{clk} is the clock frequency in hertz, and CPP is the number of clock cycles required per output pixel. At $f_{clk} = 100$ MHz and a representative $CPP = 8$, the sequential throughput is 1.25×10^7 pixels per second. This nominally exceeds the 640×480 target, but the margin shrinks quickly as resolution increases or as CPP grows with control overhead, and it leaves little headroom for higher frame rates.

7.3 Pipelined RTL Throughput

In the pipelined architecture every stage is registered and operates concurrently. Once the line buffers and pipeline registers have filled, the design accepts one input pixel and produces one output pixel on every clock cycle, so the steady-state throughput reduces to Equation (5).

$$T_{pipe} = f_{clk} \quad (5)$$

In Equation (5), T_{pipe} is the pipelined steady-state throughput in output pixels per second and f_{clk} is the clock frequency in hertz. At 100 MHz the design sustains 1.0×10^8 pixels per second, roughly ten times the requirement from Section 7.1 and an order of magnitude above the sequential estimate of Section 7.2.

Behavioural simulation confirmed this prediction. In the 128×72 test image the design consumed all 9,216 input pixels at one pixel per clock with no pipeline stalls, and produced the complete 126×70 output frame of 8,820 pixels in about 95 μ s of simulated time. That time consisted of a 2.6 μ s pipeline-fill delay followed by sustained single-cycle output. At this rate the accelerator processes more than 10,000 frames per second at this resolution. Because throughput is fixed at one pixel per clock regardless of frame size, the same design sustains real-time rates at full 640×480 resolution, about 324 frames per second. The measured steady-state throughput therefore agrees with Equation (5).

7.4 Latency Analysis

Latency is the delay between a pixel entering the accelerator and the corresponding output pixel becoming valid, and it is distinct from throughput. For this design the latency has three contributions. These are the line-buffer fill, the window-formation delay, and the arithmetic pipeline depth, as expressed in Equation (6).

$$L = 2W + D_w + D_a \quad (6)$$

In Equation (6), L is the total latency in clock cycles, W is the image width in pixels, D_w is the window-formation delay of the shift-register stage in cycles, and D_a is the depth of the arithmetic pipeline in cycles. Two full image rows ($2W = 1,280$ cycles at $W = 640$) must enter the line buffers before the first complete window column exists. Window formation adds $D_w = 3$ cycles and the two registered arithmetic stages add $D_a = 2$ cycles, for a predicted latency of 1,285 cycles.

The measured latency in simulation was exactly 1,285 clock cycles (12.85 μ s at 100 MHz) from the first pixel clocked in to the first assertion of the output valid signal, which confirms the model in Equation (6). This latency is paid once at the start of the stream and does not reduce steady-state throughput. Against a 33 ms frame period at 30 frames per second it is negligible.

7.5 Sobel Arithmetic Analysis

The Sobel operator estimates image gradients with two 3×3 kernels applied to the window pixels p_{00} through p_{22} , where p_{00} is the top-left pixel and p_{22} is the bottom-right pixel (Figure 2). The horizontal gradient is computed using Equation (7).

$$G_x = (p_{02} + 2p_{12} + p_{22}) - (p_{00} + 2p_{10} + p_{20}) \quad (7)$$

In Equation (7), G_x is the horizontal gradient estimate and the p_{ij} terms are the 8-bit pixel intensities at row i , column j of the window. The vertical gradient is computed using Equation (8).

$$G_y = (p_{20} + 2p_{21} + p_{22}) - (p_{00} + 2p_{01} + p_{02}) \quad (8)$$

In Equation (8), G_y is the vertical gradient estimate, with the p_{ij} terms defined as above. The kernel coefficients are limited to -2 , -1 , 0 , 1 , and 2 . Multiplication by two is a hard-wired left shift and multiplication by one requires no logic, so both gradients are implemented with adders, subtractors, and shifts only, avoiding DSP-block usage and reducing logic depth [2]. The edge magnitude is approximated by Equation (9).

$$M \approx |G_x| + |G_y| \quad (9)$$

In Equation (9), M is the estimated edge strength for each output pixel. The design computes this value by taking the magnitude of the left-to-right change, $|G_x|$, and the magnitude of the top-to-bottom change, $|G_y|$, then adding them together. This is simpler than calculating the exact edge strength with a square root, and the small error it introduces is acceptable for Sobel edge detection [2]. The design also leaves enough room for the numbers to grow safely during the calculation. The largest possible gradient is $4 \times 255 = 1,020$, so each gradient is stored in 13 bits. When the two gradient magnitudes are added, the largest possible value is 2,040, so that sum is stored in 14 bits before being capped to the final 8-bit pixel range. Because these widths match the largest possible values, the intermediate calculations cannot overflow.

7.6 Resource Usage Analysis

FPGA resource usage was evaluated by synthesising the RTL in Xilinx Vivado [5]. The metrics of interest are look-up tables (LUTs), flip-flops, DSP blocks, block RAM, and the maximum clock frequency from static timing analysis. Each value follows directly from the structure of the design. The adder and subtractor trees and the control logic map to LUTs, the window shift registers and pipeline stages map to flip-flops, and the two line buffers infer block RAM. Because the datapath contains no multipliers, DSP usage is zero by construction rather than by optimisation.

Synthesis confirms these expectations. The synthesized `sobel_core` netlist in Figure 4 consists of CARRY4 carry-chain adders for the G_x and G_y accumulation, LUT2 and LUT6 logic implementing the absolute-value and saturation stages, and FDRE flip-flops forming the two pipeline registers. No DSP48 primitives are present anywhere in the design. At the top level the

streaming interface is wrapped in IBUF and OBUF buffers with a BUFG on the clock, and the two line buffers map to block RAM.

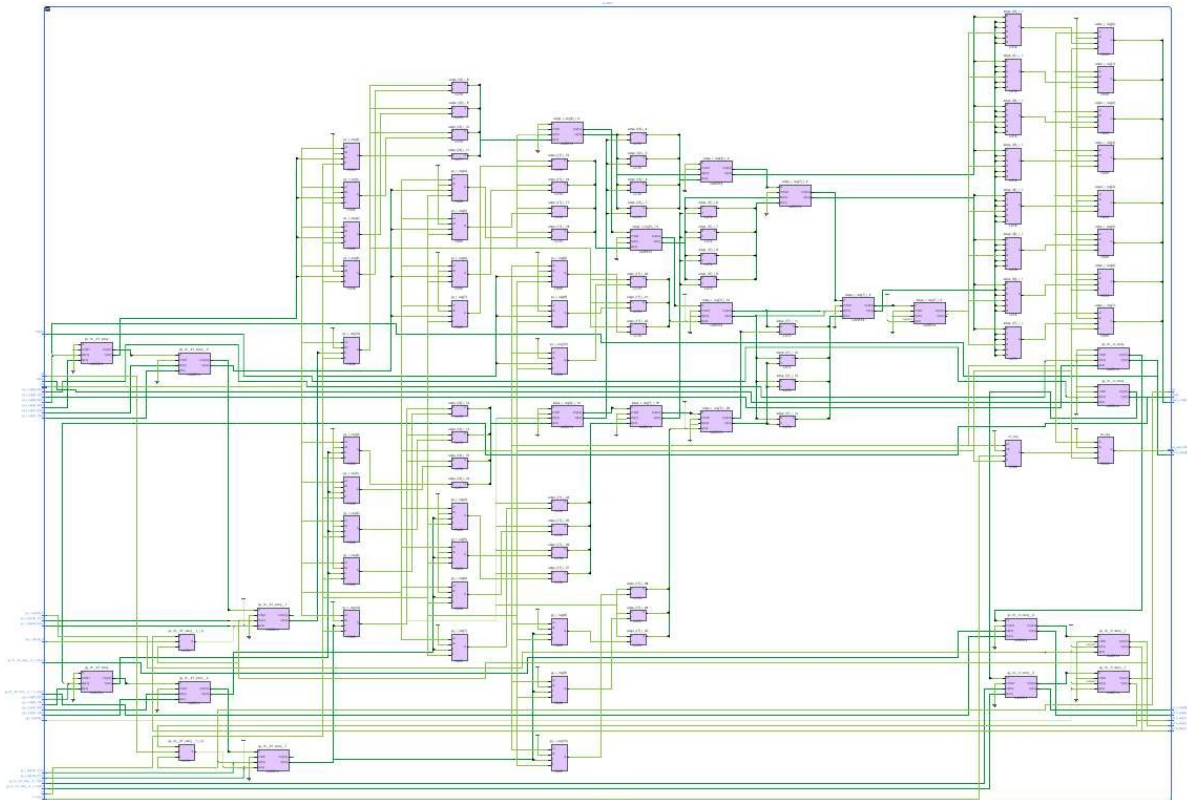


Figure 4: Synthesized netlist of the sobel_core arithmetic pipeline (Xilinx Vivado).

Power was estimated in Vivado from the synthesized netlist using vectorless activity analysis. The report gives a total on-chip power of 5.381 W, made up of 5.261 W dynamic power (98 %) and 0.120 W device static power (2 %). Within the dynamic component, signals account for 1.889 W (36 %), logic for 1.881 W (36 %), I/O for 1.403 W (26 %), and block RAM for only 0.087 W (2 %). The estimated junction temperature is 49.6 °C at a 25 °C ambient with an effective thermal resistance of 4.6 °C/W, which leaves a thermal margin of 35.4 °C (7.7 W).

These power numbers are a worst-case estimate, not the exact power the design would draw in a finished system. Vivado gives the estimate low confidence because it does not know the real switching activity of the circuit, so it makes general assumptions about how often each signal changes. In the actual design the circuit processes one pixel every clock cycle at 100 MHz, so a later estimate using real simulation activity would likely be more accurate and probably lower. Even so, the report is useful because it shows where the power goes. Most of it is used by the arithmetic logic and the I/O pins, while the line-buffer memory uses very little. This supports the two-line-buffer design choice, and using no DSP blocks keeps that part of the power at zero.

7.7 Verification Results

The design was verified by behavioural simulation. A conversion script turns a source image into a text stream of pixel values, the VHDL testbench streams the pixels through the design one per clock, and the recorded output stream is rebuilt into an image for inspection. Simulation runs were performed at several frame sizes, including the two photographs shown below and a full 640×480 frame.

Every run produced the expected output frame of $(W - 2) \times (H - 2)$ pixels with no missing or extra pixels. In the 128×72 run this was the complete 126×70 frame of 8,820 pixels. Reset and border behaviour are exercised by every run, since each simulation starts from reset and the valid logic must suppress the border pixels for which no complete 3×3 window exists. The timing of the output valid signal also matched the latency model of Section 7.4 exactly.

Figures 5 and 6 show representative results for two photographs, a 128×72 image of a car and a 228×128 image of two people. In each case the edge image clearly outlines the objects in the scene.



Figure 5: Sobel verification result for a 128×72 input image.



Figure 6: Sobel verification result for a 228×128 input image.

7.8 Sensitivity and Limitations of the Analysis

The real-time margin is sensitive to three parameters. From Equations (3) and (5), the margin is the ratio of f_{clk} to $W \times H \times F$. Raising the resolution or frame rate increases the requirement linearly, while a lower achievable clock frequency reduces the supply. The sequential alternative also scales inversely with CPP from Equation (4), which makes it far more sensitive to control overhead. The pipelined design keeps a large margin across plausible operating points.

The analysis itself has limits that should be kept in view. It covers grayscale images and the fixed 3×3 Sobel operator only. Larger kernels would add line buffers and arithmetic depth, and RGB processing would widen or multiplex the datapath. The throughput and latency figures assume an ideal stall-free input stream, so a real camera or bus interface with back-pressure would need the ready/valid handshake noted in the proposed solution. Synthesis confirms the hardware structure and the absence of DSP blocks, but the post-implementation timing report and a power estimate based on real switching activity remain future work, so the 100 MHz operating point rests on the shallow pipeline rather than a signed-off timing analysis. Finally, verification rests on structural

and visual checks of simulation outputs across a limited set of test images, so it demonstrates correct operation on those images rather than proving correctness in general. It also does not evaluate how well the Sobel operator itself detects edges, which is a property of the algorithm rather than of this design.

8.0 Discussion

The measurements support the choice of the pipelined architecture. The design sustains one output pixel per clock cycle in simulation, which at 100 MHz is about ten times the 9.216×10^6 pixels-per-second requirement derived in Section 7.1 and an order of magnitude beyond the sequential estimate. Throughput is the quantity most directly tied to the engineering problem of deterministic real-time processing, so this margin is the decisive result.

The cost of that margin is modest. The extra hardware is dominated by the two line buffers, which fit in block RAM, and by pipeline registers. The arithmetic uses no multipliers at all. The design therefore trades a small, practical amount of memory for a tenfold throughput margin, which is a favourable exchange on a modern FPGA. The power analysis points the same way. The line-buffer memory draws only 2 % of the estimated on-chip power, so the memory that buys the throughput margin costs almost nothing in either area or power.

The exact agreement between the predicted and measured latency (1,285 cycles in both cases) is worth emphasising, because it validates the analytical model of Equation (6) rather than just the simulation. Latency and throughput remain distinct quantities. The $12.85 \mu\text{s}$ fill delay is paid once per stream and is negligible against a 33 ms frame period, although a latency-critical closed-loop application would still need to budget for it explicitly.

The main remaining limitation is the scope of verification. The design has been exercised in simulation across several image sizes, up to and including a full 640×480 frame, and every run produced the expected output frame and a clean edge image. This provides strong practical confidence in the RTL, but it is not a formal proof of correctness. The Vivado synthesis results also confirm the intended hardware structure. The arithmetic maps to carry-chain adders, LUT logic, and pipeline registers, with no DSP blocks used, and the vectorless power figure of 5.4 W stands as a worst-case bound until a switching-activity-based estimate replaces it.

Beyond the immediate project, the results generalise in useful ways. The measured 100 Mpixel/s capability already exceeds the 62.2×10^6 pixels-per-second demand of 1080p video at 30 frames per second, so the same architecture supports full-HD edge detection with no structural change beyond longer line buffers. The line-buffer-and-window pattern is not specific to Sobel. Any 3×3 kernel, including Gaussian smoothing or sharpening, can reuse the same streaming structure, so both the design and the analysis method transfer directly to other streaming vision accelerators. Future work could add an AXI-Stream wrapper and post-implementation switching-activity annotation to integrate and characterise the block in a larger video pipeline.

9.0 Conclusions

The pipelined RTL filter is the most suitable architecture for the real-time Sobel accelerator, and the purpose of the report has been met. The primary aim is satisfied with a wide margin. The design sustained one output pixel per clock cycle in simulation, equivalent to 100 million pixels per second at a 100 MHz clock against a requirement of 9.216 million pixels per second, and a full 640×480 frame completes in 3.085 ms, or about 324 frames per second.

The design behaves correctly in simulation. Runs across several image sizes, up to and including the full 640×480 frame, produced the expected output frames and clean edge images. The measured pipeline latency of 1,285 clock cycles agreed exactly with the analytical prediction, validating the latency model as well as the implementation.

The pipelined design outperforms the sequential alternative by roughly an order of magnitude on the cycles-per-pixel metric. Its arithmetic is implemented entirely with adders, subtractors, and shifts, and Vivado synthesis confirms that the netlist contains no DSP blocks and that the two line buffers map to block RAM. The vectorless power estimate of 5.4 W is a worst-case bound that a switching-activity-based analysis would likely reduce. A signed-off post-implementation timing report remains outstanding, and all conclusions hold within the stated limitations of grayscale 3×3 filtering and an idealised stall-free pixel stream.

10.0 Recommendations

1. Run Vivado implementation and static timing analysis to confirm timing closure at 100 MHz and record the achieved clock frequency.
2. Re-estimate power from post-implementation simulation with recorded switching activity to replace the worst-case vectorless figure.
3. Add an AXI-Stream ready/valid wrapper so the accelerator interoperates with back-pressured streaming systems.
4. Extend the design to RGB images and configurable kernel coefficients to broaden its applicability.
5. Benchmark the accelerator against CPU and high-level-synthesis implementations to quantify the benefit.
6. Strengthen verification with randomised input images and assertion-based checks on the valid-signal timing.

References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY: Pearson, 2018.

- [2] N. Kanopoulos, N. Vasanthavada, and R. L. Baker, "Design of an image edge detection filter using the Sobel operator," *IEEE J. Solid-State Circuits*, Vol. 23, No. 2, pp. 358–367, Apr. 1988.
- [3] D. G. Bailey, *Design for Embedded Image Processing on FPGAs*. Singapore: John Wiley & Sons (Asia), 2011.
- [4] *Vivado Design Suite User Guide: Logic Simulation*, UG900 (v2024.1), Advanced Micro Devices, Inc., San Jose, CA, 2024.
- [5] *Vivado Design Suite User Guide: Synthesis*, UG901 (v2024.1), Advanced Micro Devices, Inc., San Jose, CA, 2024.

Appendix A: Supporting Calculations

This appendix presents the worked calculations summarised in Section 7.0. The project parameters are $W = 640$, $H = 480$, $F = 30$ frames per second, and $f_{clk} = 100$ MHz, with a 128×72 test image used for the detailed simulation run.

A.1 Required Pixel Rate

$$R_{px} = W \times H \times F = 640 \times 480 \times 30 = 9,216,000 \text{ pixels per second.}$$

A.2 Sequential Throughput Estimate

$$T_{seq} = f_{clk} / CPP = 100 \times 10^6 / 8 = 12.5 \times 10^6 \text{ pixels per second, a margin of only } 12.5 / 9.216 \approx 1.36 \text{ over the requirement.}$$

A.3 Pipelined Throughput and Margin

$T_{pipe} = f_{clk} = 100 \times 10^6$ pixels per second, a margin of $100 / 9.216 \approx 10.9$ over the requirement. For 1080p at 30 frames per second the demand is $1920 \times 1080 \times 30 = 62.2 \times 10^6$ pixels per second, still within the measured capability.

A.4 Latency

At $W = 640$ the prediction is $L = 2W + D_w + D_a = 1,280 + 3 + 2 = 1,285$ cycles, which is $12.85 \mu\text{s}$ at 10 ns per cycle. The measured value was 1,285 cycles, matching the prediction exactly. For the 128×72 test image the same model gives $L = 256 + 3 + 2 = 261$ cycles, which is the $2.6 \mu\text{s}$ fill delay observed in that run.

A.5 Frame Time and Frame Rate

A full 640×480 frame requires the 307,200 input pixels plus the fill latency, so $(307,200 + 1,285) \times 10 \text{ ns} \approx 3.085 \text{ ms}$, giving about 324 frames per second. The 128×72 run requires $(9,216 + 261) \times 10 \text{ ns} \approx 95 \mu\text{s}$, which is more than 10,000 frames per second at that resolution.

A.6 Bit-Width Bounds

The largest weighted sum in either gradient is $(255 + 2 \times 255 + 255) = 1,020$, so G_x and G_y fit in 13-bit signed words. The magnitude sum $|G_x| + |G_y|$ is at most 2,040, held in 14 unsigned bits before saturation to the 8-bit output range.

A.7 Line-Buffer Memory

Each line buffer stores one image row of $W \times 8 \text{ bits} = 640 \times 8 = 5,120 \text{ bits}$, which fits in a single 18 Kb block RAM. The design therefore needs two block RAMs in total.